

TRAVELING SALESMAN PROBLEM USING METAHEURISTIC ALGORITHMS

Project Report

Group: *Oswalds*

Student Names and IDs:

Arlene Riona Devasahayarajan – 60304739

Fariha Aslam Mahaldar - 60306249

Syeda Maymona Mustafa - 60306027

Mais Mardoum - 60308505

Zahra Alwaili - 60307052

Instructor: *Dr. Somaiyeh M. Zadeh*

University of Doha for Science and Technology

13th April 2026

Contents

1. Introduction.....	4
1.2 Overview of the Problem and Problem Statement.....	4
1.2.1 Problem Statement.....	4
1.2.3 Motivation and Importance of the Study	5
2. Mathematical Formulation of the Travelling Salesman Problem.....	5
2.1 Problem Definition.....	5
2.2 Formal Graph-Theoretic Formulation.....	5
2.2.1 Basic Definition	5
2.2.2 Matrix Representation.....	5
2.3 Binary Integer Programming Formulation.....	6
2.3.1 Decision Variables	6
2.3.2 Objective Function.....	7
2.3.3 Constraints	7
2.4 Computational Complexity and Hardness	8
2.4.1 NP-Hardness Classification	8
2.4.2 Inapproximability and Practical Implications	8
3. Overview of the Chosen Algorithms	9
3.1 Traveling Salesman Problem (TSP) Formulation.....	9
3.1.1 Solution Representation	9
3.1.2 Challenges of the TSP Problem	10
3.2 Adaptive Grey Wolf Optimizer (AGWO)	10
3.2.1 Inspiration and Concept	10
3.2.2 Mathematical Model	10
3.2.3 Adaptive Mechanism	11
3.2.4 Adaptation to TSP.....	11
3.3 Arithmetic Optimization Algorithm (AOA).....	12
3.3.1 Inspiration and Concept	12
3.3.2 Search Mechanism	12
3.3.3 Mathematical Model	12
3.3.4 Adaptation to TSP.....	12

4. Methodology	13
4.1 Problem Modeling:	13
4.2 Benchmark Dataset	14
4.3 Experimental Setup	14
5. Results and Discussion	17
5.2 Experimental Results	17
5.3 Analysis of Results	18
5.4 Limitations of the Results	21
5.5 Possible Improvements	22
Conclusion	23

1. Introduction

Combinatorial optimization is an important area in computer science and applied mathematics, focusing on problems where the goal is to find the best solution from a large number of possible combinations. One of the most well-known problems in this field is the Traveling Salesman Problem (TSP). The problem is simple to describe given a set of cities and the distances between them. The aim is to find the shortest possible route that visits each city exactly once and returns to the starting point. However, despite its simple definition, TSP is extremely challenging to solve, especially as the number of cities increases. This is because the number of possible routes grows very quickly (factorially), making exact solutions impractical for larger instances.

Due to this complexity, researchers have focused on developing approximate methods that can produce good solutions within a reasonable time. These methods, known as metaheuristic algorithms, do not always guarantee optimal solutions but are effective for solving large-scale problems.

The importance of TSP is not limited to theory. It has many real-world applications, particularly in logistics and transportation, where efficient route planning can reduce operational costs, improve delivery times, and minimize environmental impact. Small improvements in route efficiency can lead to significant savings when applied at scale.

In this project, two metaheuristic algorithms are explored: the Adaptive Grey Wolf Optimizer (AGWO) and the Arithmetic Optimization Algorithm (AOA). These algorithms represent different approaches to solving optimization problems—one inspired by natural behavior and the other based on mathematical operations. The main aim of this study is to compare their performance on TSP instances and analyze their effectiveness based on solution quality and convergence behavior.

1.2 Overview of the Problem and Problem Statement

1.2.1 Problem Statement

The Traveling Salesman Problem (TSP) can be formally defined as follows: given a set of n cities and a distance matrix that represents the distance between each pair of cities, the objective is to find a route (also known as a Hamiltonian cycle) that visits each city exactly once and minimizes the total travel distance.

TSP is classified as an NP-hard problem, which means that there is no known algorithm that can solve all instances efficiently as the problem size increases. For small numbers of cities (for example, fewer than 20), exact algorithms can be used to find the optimal solution. However, as the number of cities grows, the number of possible routes increases dramatically. For example, a 52-city TSP instance has more than 10^{67} possible routes, making it impossible to evaluate all solutions within a reasonable time.

1.2.3 Motivation and Importance of the Study

The Traveling Salesman Problem is important due to both its theoretical complexity and its practical applications. In real-world scenarios such as logistics and delivery systems, efficient route planning is essential for reducing costs and improving performance. Since exact solutions are not feasible for large problem sizes, approximate methods are commonly used.

This study focuses on two metaheuristic algorithms, the Adaptive Grey Wolf Optimizer (AGWO) and the Arithmetic Optimization Algorithm (AOA), to evaluate their effectiveness in solving TSP. By comparing their performance, this project aims to identify which approach provides better solutions under similar conditions.

2. Mathematical Formulation of the Travelling Salesman Problem

2.1 Problem Definition

The Traveling Salesman Problem (TSP) is one of the most studied problems in combinatorial optimization and theoretical computer science. In its simplest form, the TSP can be stated as follows: given a finite set of cities and the distances between each pair of cities, find the shortest route that visits each city exactly once and returns to the starting city (De Giovanni & Di Summa, 2018). Despite the simplicity of this statement, the problem is difficult, with implications for both theoretical research and practical applications.

The mathematical study of TSP has led to significant advances in optimization theory and practice. Integer programming formulations have been developed with increasingly tight bounds, polyhedral theory has been applied to understand the geometric structure of the solution space, and sophisticated computational techniques have enabled researchers to solve instances with thousands of cities (De Giovanni & Di Summa, 2018). For a general instance of TSP with n cities, no algorithm is known that can guarantee finding the optimal solution in time polynomial in n .

2.2 Formal Graph-Theoretic Formulation

2.2.1 Basic Definition

The TSP can be defined on a complete weighted graph as follows:

Let $G = (V, E)$, be a complete undirected graph where:

$V = \{1, 2, 3, \dots, n\}$ is the set of n vertices

E is the set of edges connecting every pair of distinct vertices, where $|E| = \frac{n(n-1)}{2}$

$d(i, j)$ represents the Euclidean distance between city i and j .

The problem requires finding a Hamiltonian cycle, which is a closed path that visits every vertex in V exactly once, with minimum total edge weight.

The Euclidean distance between any two cities i and j with coordinates (x_i, y_i) and (x_j, y_j) is computed as:

$$d(i, j) = \sqrt{[(x_j - x_i)^2 + (y_j - y_i)^2]}$$

2.2.2 Matrix Representation

For a TSP instance with n cities, all pairwise distances are compactly stored in a $n \times n$ distance matrix D , where element d_{ij} , denotes the distance from city i to city j . The following table shows a sample 5×5 distance matrix for a five-city TSP instance:

	City 1	City 2	City 3	City 4	City 5
City 1	0	12	30	25	18
City 2	12	0	15	28	22
City 3	20	15	0	10	30
City 4	35	28	18	0	25
City 5	18	22	30	25	0

Table 1: A 5×5 distance matrix D for a five-city TSP instance. All diagonal entries are 0 as a city has no distance to itself.

2.3 Binary Integer Programming Formulation

2.3.1 Decision Variables

The most common exact mathematical formulation of TSP uses binary integer programming. A binary decision variable x_{ij} is defined for each ordered pair of distinct cities as follows:

$x_{ij} = 1$ if the salesman travels directly from city i to city j , $x_{ij} = 0$ otherwise

where $i, j \in \{1, 2, \dots, n\}, i \neq j$ and $x_{ij} \in \{0, 1\}$

2.3.2 Objective Function

The goal is to minimize the total distance of the Hamiltonian cycle. The objective function sums the distances of all edges selected for inclusion in the tour.

$$\text{Minimize } Z = \sum_{i=1}^n \sum_{j=1, j \neq i}^n d_{ij} \cdot x_{ij}$$

The coefficient d_{ij} weights the contribution of each edge to total cost. The optimization seeks an assignment of x_{ij} values such that the resulting tour has the minimum possible total distance while satisfying all feasibility constraints (De Giovanni & Di Summa, 2018).

2.3.3 Constraints

The objective function alone does not guarantee a valid tour. Three sets of constraints are imposed to ensure feasibility:

Constraint 1: Degree Constraints (Leaving Each City)

$$\sum_{j=1, j \neq i}^n x_{ij} = 1, \quad \forall i \in V$$

For each city i , exactly one outgoing edge must be selected, ensuring the tour departs from every city to precisely one other city.

Example for a 5-city problem: City 1, if $x_{12} = 1$, then $x_{13} = x_{14} = x_{15} = 0$, meaning the tour leaves city 1 and proceeds directly to city 2.

Constraint 2: Degree Constraints (Entering Each City)

$$\sum_{i=1, i \neq j}^n x_{ij} = 1 \quad \forall j \in V$$

For each city j , incoming edge must be selected, ensuring each city is visited exactly once from exactly one predecessor.

Example for a 5-city problem: City 3, if $x_{13} = 1$, then $x_{23} = x_{43} = x_{53} = 0$, meaning only city 1 has an edge directed into city 3.

Combined effect of Constraints 1 and 2: Together, these ensure each city has in-degree 1 and out-degree 1, which is a necessary condition for a Hamiltonian cycle. However, they are insufficient alone as they permit multiple disconnected subtours, each satisfying both constraints but not forming a single valid tour.

Constraint 3: Subtour Elimination (Miller-Tucker-Zemlin Formulation).

$$u_i - u_j + n \cdot x_{ij} \leq n - 1, \forall i, j \in V, i \neq j$$

where u_i is a continuous variable representing the visit sequence of city i .

This constraint enforces that the selected edges form a single connected Hamiltonian cycle rather than multiple disconnected subtours. The auxiliary variable u_i encodes the position of city i in the tour sequence.

How it works:

- If $x_{ij} = 0$ (edge (i, j) is not in the tour), the constraint becomes $u_i - u_j \leq n - 1$, which is always satisfied since $1 \leq u_i, u_j \leq n$.
- If $x_{ij} = 1$ (edge (i, j) is in the tour), the constraint becomes $u_i - u_j + n \leq n - 1$, which simplifies to $u_j \geq u_i + 1$. This forces city j to be visited strictly after city i in the sequence, preventing any cycle from forming among a subset of cities.

2.4 Computational Complexity and Hardness

2.4.1 NP-Hardness Classification

The Traveling Salesman Problem belongs to the class of NP-hard problems, a classification with profound implications for how the problem must be computationally approached. Understanding this classification is essential to appreciating why metaheuristic algorithms such as AGWO and AOA are used in this report.

A problem is NP-hard if every problem in the class NP can be reduced to it in polynomial time. If a polynomial-time algorithm were discovered for any NP-hard problem, it would imply polynomial-time algorithms exist for all problems in NP, which results in a widely believed to be impossible solution (Toaza & Esztergár-Kiss, 2023).

The decision version of TSP for "Does there exist a tour with total cost at most K ?", is NP-complete. The optimization version, "Find the tour with minimum total cost", is NP-hard. No polynomial-time exact algorithm is known for either formulation (De Giovanni & Di Summa, 2018).

2.4.2 Inapproximability and Practical Implications

Beyond NP-hardness, general TSP is also inapproximable to any constant factor in polynomial time unless $P = NP$ (Toaza & Esztergár-Kiss, 2023). This means no efficient algorithm can guarantee a solution within a fixed percentage of optimal for all instances. For practical TSP instances that satisfy Metric TSP, a 1.5-approximation is achievable via the

Christofides algorithm; however, for real-world large-scale instances, this bound is rarely tight enough to be operationally useful (Vidal, 2022).

These theoretical barriers justify the use of metaheuristic approaches. While metaheuristics such as the Adapted Grey Wolf Optimizer (AGWO) and Arithmetic Optimization Algorithm (AOA) do not guarantee globally optimal solutions, they are designed to produce high-quality approximate solutions within practical computational time, making them well-suited to TSP at realistic scales.

3. Overview of the Chosen Algorithms

3.1 Traveling Salesman Problem (TSP) Formulation

The Traveling Salesman Problem (TSP) is a classical combinatorial optimization problem in which the objective is to determine the shortest possible route that visits a set of n exactly once and returns to the starting city. Let $V = \{1, 2, \dots, n\}$ represent the set of cities and d_{ij} represent the distance between city i and city j . The goal is to construct a Hamiltonian cycle that minimizes total travel distance.

$$\min \sum_{i=1}^n \sum_{j=1}^n d_{ij} x_{ij}$$

subject to:

- each city is visited exactly once
- each city has exactly one incoming and one outgoing edge
- the solution forms a single closed tour

Where $x_{ij} \in \{0, 1\}$ indicates whether a path between city i and j is selected.

3.1.1 Solution Representation

In this study, each candidate solution is represented as a permutation of cities:

$$X = [c_1, c_2, c_3, \dots, c_n]$$

This representation ensures that each city is visited exactly once. The fitness function evaluates the quality of a solution by computing the total travel distance:

$$f(X) = \sum_{i=1}^{n-1} d(c_i, c_{i+1}) + d(c_n, c_1)$$

The objective is to minimize this value, thereby identifying the shortest possible tour.

3.1.2 Challenges of the TSP Problem

The TSP is classified as an NP-hard problem due to the factorial growth of possible solutions. The number of possible tours increases as $(n - 1)!$, making exhaustive search impractical even for moderately sized problems.

For example:

- 10 cities $\rightarrow$ 3.6 million possible tours
- 15 cities $\rightarrow$ over 87 billion possible tours

This leads to several significant challenges:

- Combinatorial explosion, resulting in a vast search space
- Discrete solution structure, requiring permutation-based representations
- Susceptibility to local optima, where simple heuristics fail
- High computational complexity for exact algorithms

Because of these challenges, metaheuristic algorithms such as AGWO and AOA are employed to efficiently explore the search space and produce high-quality approximate solutions (Mirjalili et al., 2014; Abualigah et al., 2021).

3.2 Adaptive Grey Wolf Optimizer (AGWO)

3.2.1 Inspiration and Concept

The Adaptive Grey Wolf Optimizer (AGWO) is an enhanced version of the Grey Wolf Optimizer (GWO), originally proposed by Mirjalili et al. (2014). It is inspired by the hierarchical leadership and cooperative hunting strategy of grey wolves. In this hierarchy, the best solution is considered the alpha, followed by beta and delta, while the remaining solutions are categorized as omega. These top three solutions guide the search process toward optimal regions.

3.2.2 Mathematical Model

The encircling mechanism is defined as:

$$D = |C \cdot X_p(t) - X(t)|$$

$$X(t + 1) = X_p(t) - A \cdot D$$

where A and C control the balance between exploration and exploitation.

3.2.3 Adaptive Mechanism

AGWO improves the original model by dynamically adjusting parameter A allowing a gradual transition from global exploration to local exploitation. This adaptive behavior enhances convergence speed and reduces the likelihood of getting trapped in local optima (Mohamed et al., 2021).

3.2.4 Adaptation to TSP

To apply AGWO to the TSP, the algorithm must be adapted from a continuous search space to a discrete permutation-based representation. In this context, each wolf represents a candidate tour, encoded as an ordered sequence of cities. However, the original position update equations produce continuous values, which are not directly applicable to permutations.

To address this, the continuous updates are interpreted as guidance signals rather than direct position values. The influence of the alpha, beta, and delta wolves is used to modify existing tours through discrete operators. Specifically, elements from the best solutions are incorporated into other candidate solutions using operations such as swapping city positions, reversing subsequences (inversion), or partially reconstructing routes based on the structure of the best-performing tours.

Additionally, after each update, a repair mechanism ensures that the resulting solution remains a valid permutation (i.e., no duplicate cities and all cities are included). This adaptation enables AGWO to effectively search within the discrete solution space of TSP while preserving its exploration and exploitation balance.

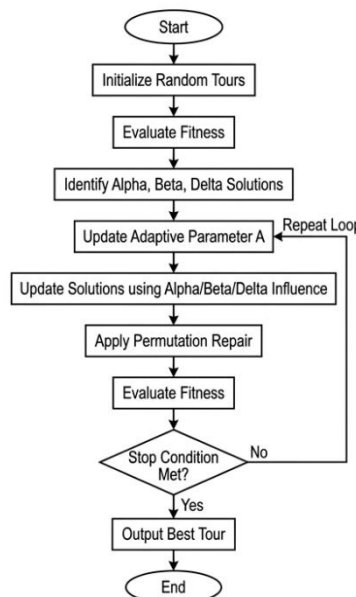


Figure 1: Flowchart of the Adaptive Grey Wolf Optimizer (AGWO) applied to the Traveling Salesman Problem.

3.3 Arithmetic Optimization Algorithm (AOA)

3.3.1 Inspiration and Concept

The Arithmetic Optimization Algorithm (AOA) is a population-based optimization method inspired by basic arithmetic operations. It uses addition, subtraction, multiplication, and division to control the movement of solutions in the search space (Abualigah et al., 2021).

3.3.2 Search Mechanism

AOA uses the Math Optimizer Accelerated (MOA) parameter to determine whether the algorithm performs exploration or exploitation. Multiplication and division promote diversification, while addition and subtraction refine existing solutions.

3.3.3 Mathematical Model

Exploitation:

$$X_i^{t+1} = X_i^t \pm r(X_{best} - X_i^t)$$

Exploration:

$$X_i^{t+1} = X_i^t \cdot r \text{ or } X_i^t \div (r + \epsilon)$$

3.3.4 Adaptation to TSP

Like AGWO, AOA must be adapted to handle the discrete nature of TSP. Each candidate's solution is represented as a permutation of cities, and arithmetic operations are reinterpreted to operate on these permutations.

Instead of directly applying numerical operations, the effect of arithmetic operators is translated into structural modifications of the tour. For example, multiplication and division operators, which represent exploration, may correspond to large changes in the tour, such as reversing segments or randomly reshuffling parts of the route. In contrast, addition and subtraction operators, which represent exploitation, are implemented as smaller, localized adjustments such as swapping adjacent cities or inserting a city into a different position.

Furthermore, the best solution identified so far guides the modification process, ensuring that new solutions move toward promising regions of the search space. After each update, a repair mechanism is applied to maintain valid permutations. This adaptation allows AOA to effectively explore and refine solutions within the discrete domain of TSP.

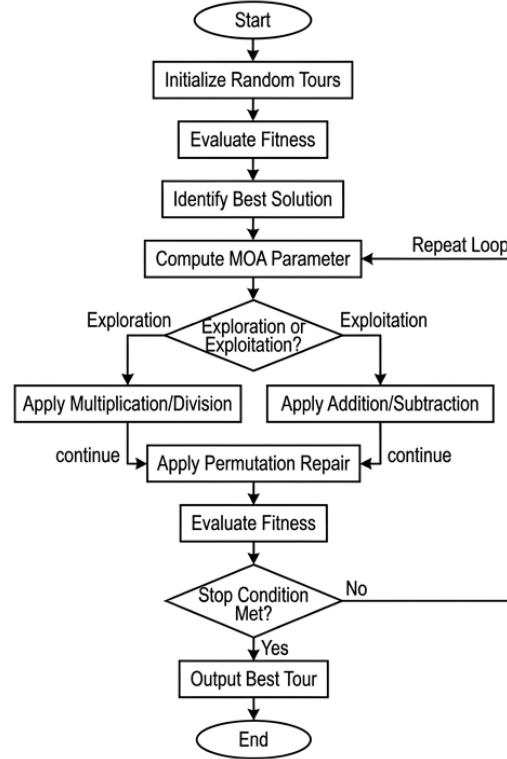


Figure 2: Flowchart of the Arithmetic Optimization Algorithm (AOA) applied to the Traveling Salesman Problem.

4. Methodology

The Traveling Salesman Problem (TSP) seeks the shortest closed tour that visits each city once and returns to the starting city (Xu and Zhang, 2024). It is a Non-deterministic Polynomial time hard problem (NP-Hard) where the number of possible solutions increases with the factorial of n , thus becoming unfeasible (Toaza and Esztergár-Kiss, 2023). Therefore, heuristic algorithms are required to obtain near-optimal solutions. In this work, the algorithms of Arithmetic Optimization Algorithm (AOA) and Adaptive Grey Wolf Optimizer (AGWO) we applied.

4.1 Problem Modeling:

The Travelling Salesman Problem is modeled as follows:

Each city is modeled as a node with two-dimensional cartesian coordinates (x, y) , which represents its position in Euclidean space. These coordinates are used to calculate the pairwise distance between cities.;

The distance between cities is considered the travelling cost; it is defined using basic Euclidean distance metric, as given in equation below.

$$d(i, j) = \sqrt{(x_i - x_j)^2 + (y_i - y_j)^2}$$

Where $i = (x_i, y_i)$ and $j = (x_j, y_j)$ represent the locations of cities i and j , respectively.

A complete solution to the problem, referred to as tour, is represented as a permutation of all cities (nodes). Where each city is visited exactly once, and all cities are visited before returning to the starting city, as shown:

$$T = [C_3, C_5, C_1, C_2, \dots]$$

The fitness value of a candidate solution, which we were aiming to minimize, was calculated as the total distance of the tour, using the below equation. Reducing fitness value means shorter distance and less costs.

$$f(T) = \sum_{k=1}^{n-1} d(C_k, C_{k+1}) + d(C_n, C_1)$$

4.2 Benchmark Dataset

In our experiment, we used the Kaggle TSPLIB datasets, the berlin52 instance (Ziya, 2025). It contains 52 cities' coordinates. It has a known optimal tour length value of 7,542 distance units that we aim to reach. We chose this dataset because it is moderately sized, where it is not too small and allows multiple iterations; yet not too large, so we can still reach a near-optimal value. This dataset was used in multiple similar research papers applying similar algorithms such as: D-GWO, adapting genetic algorithms to GWO (Xu and Zhang, 2024).

4.3 Experimental Setup

To ensure fair comparison between Arithmetic Optimization Algorithm (AOA) and Adaptive Grey Wolf Optimizer (AGWO), we set a consistent framework as shown in Table 2.2

<i>Parameter</i>	Value
<i>Number of runs</i>	10
<i>Population size</i>	30 agents/wolves per run
<i>Maximum iterations per run</i>	500
<i>Post-Processing</i>	2-opt local search
<i>Random seeds</i>	1-10 (Different per run)

Table 2: Experimental Configuration Parameters

Each algorithm was run 10 times, each with different seeds to enable statistical analysis of performance consistency. Population size was set to 30 candidate solutions to balance the search process and computational efficiency. We set the number of iterations to 500 to provide sufficient number of iterations for convergence and avoid too many iterations. The 2-opt refinement was applied at the end to preserve the distinct search characteristics while benefiting local optimization.

4.4 Algorithm Implementation

4.4.1 Largest Order Value (LOV) Permutation

Both AOA and AGWO operate on continuous solution spaces; the Largest Order Value (LOV) method was applied to convert continuous vectors into valid city permutations. It sorts elements in a continuous vector $X = [x_1, x_2, x_3, \dots, x_n]$, where each $x_i \in [0,1)$, into descending order. It then uses the indices of the sorted positions as the city visitation order. This guarantees that each city appears exactly once on the resulting tour.

4.4.2 Arithmetic Optimization Algorithm (AOA) Implementation

The population was initialized by generating random candidate solutions of city indices, representing the initialization phase. Each tour was evaluated using the Euclidean distance fitness function, and the best solution was identified. At each iteration, the Math Optimizer Probability (MOA) is calculated using:

$$MOA(t) = 0.1 + t \times \frac{0.9}{T}$$

If a random number exceeded the current MOA value, the algorithm randomly swaps two cities in the tour, which allows the algorithm to explore new regions of the search space; this is the exploration phase. When a random number is less than or equal to the MOA value, the algorithm removes a randomly selected city and inserts it at a different position in the tour; this is the exploitation phase.

After each update, the new tour gets evaluated using the fitness value. If it was better than the current best solution, the best solution would be replaced. After finishing all 500 iterations, a 2-opt heuristic local search is applied to the best solution found to further improve the solution quality.

4.4.3 Adaptive Grey Wolf Optimizer (AGWO) Implementation

The population was initialized by generating random candidate solutions (wolves), each representing a complete tour of all cities. Each wolf was evaluated using the fitness function (total Euclidean distance), and the three wolves with the best fitness were designated as alpha (α), beta (β), and delta (δ), representing the leadership hierarchy.

At each iteration, an adaptive parameter a was calculated using:

$$a = 2 \times \left(1 - \left(\frac{t}{T}\right)^2\right)$$

For each wolf, a guide wolf is selected to update its position using order crossover, which is a segment copied from the guide wolf into the same positions. The remaining positions were filled with cities from the current wolf in the order they appeared.

An adaptive mutation was applied with probability $1 - \frac{t}{T}$, which decreases over time. When triggered, two random cities in the tour get swapped, encouraging exploration early in the search and exploitation later.

After each update, the new tour is evaluated using the fitness function. If it was better than the current wolf's fitness, the wolf gets updated. Finally, a 2-opt heuristic local search was applied to the alpha (best) solution to finalize the solution.

4.4.4 Post-Processing: 2-Opt Local Search

After completing all 500 iterations, a 2-opt local search heuristic is applied to the final best solution. The 2-opt algorithm examines the edges of the tour and reverses the segment between them to reduce the total distance if possible. This process gets repeated until no further improvements can be made. The 2-opt refinement is only applied at termination.

4.5 Performance Metrics

To enable comprehensive between AOA and AGWO, the following metrics were recorded for each algorithm throughout the 10 runs:

- **Best Distance:** The minimum tour distance across all runs.
- **Worst Distance:** The maximum tour distance across all runs.
- **Mean Distance:** The average tour distance across all runs (indicates the overall performance).
- **Standard Deviation:** To measure consistency; lower values indicate more reliable performance.
- **Average Runtime:** The mean execution time per run, in seconds.
- **Optimality Gap:** The percentage difference between the best final solution and the known optimal value (7,542), calculated by:

$$Gap = \frac{D_{best} - D_{optimal}}{D_{optimal}} \times 100\%$$

5. Results and Discussion

This section presents the experimental results obtained from running AOA and AGWO on the berlin52 benchmark, followed by a detailed analysis of each figure, statistical robustness evaluation, identified limitations, and proposed improvements, In this part we evaluated our chosen algorithms AOA and AGWO on the **berlin52** benchmark instance from the **TSPLIB** dataset, which has 52 cities and the known optimal length of **7,542 units**, we executed every algorithm for 10 runs using 30 agents over 500 iterations , and then to refine the final tour we followed with a 2-opt local search post-processing, finally we evaluated both algorithms using the best distance, worst distance , standard deviation, optimality gap and average run time.

5.2 Experimental Results

Algorithm	Cities	Best Dist.	Avg. Dist.	Worst Dist.	Std Dev (σ)	Avg Runtime (s)	Gap %
AOA	52	7742.6	8351.6	8720.8	250.4	7.87	2.66%
AGWO	52	8001.9	8271.1	8781.0	201.4	11.92	6.10%
Optimal	52	7542.0	—	—	—	—	0%

Table 3: Performance Comparison of AOA and AGWO

In this table we can see the clear trade off between risk and reward, AOA was able to find the best overall route which is (7742.6) but it has a high standard deviation which means it's a hit or miss algorithm, while on the other hand AGWO showed more consistent results across all the iterations which make it generally more reliable, AOA is only better to find a single golden solution but if we are aiming for a safer and steady performance AGWO will be the answer.

5.3 Analysis of Results

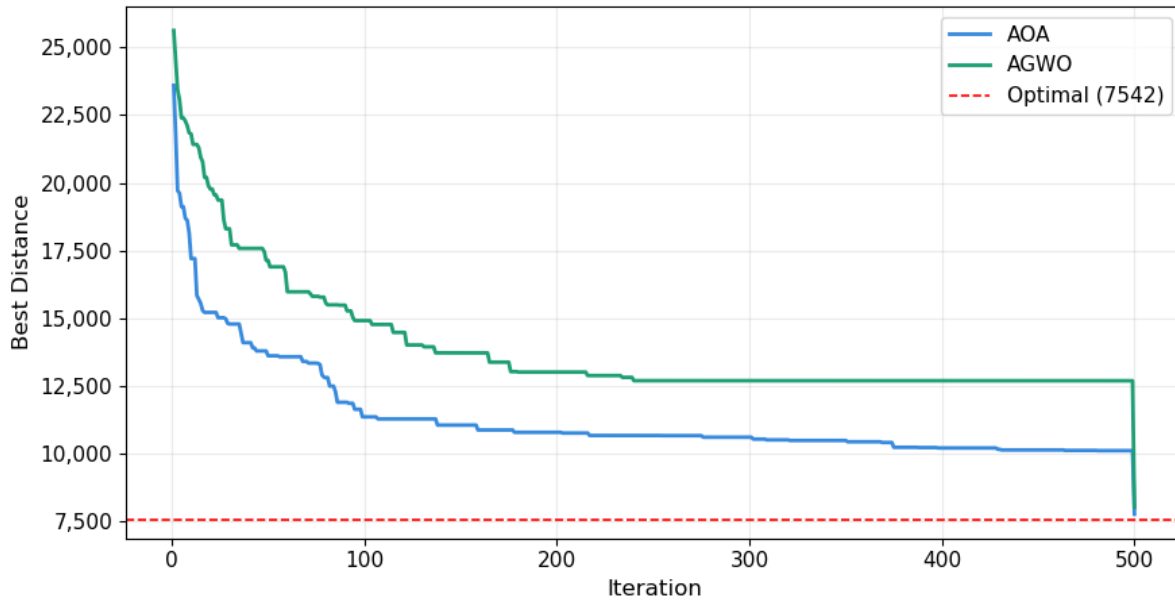


Figure 3: convergence curves – AOA vs AGWO (berlin 25)

Convergence Behavior (Figure 3): In this graph the convergence curves shows the difference in the searching mechanisms between the two methods over 500 iterations, AOA has a more rapid convergent at the early iterations as it is using the simple arithmetic operators consuming less computational power and no population wide sorting quickly reducing its best distance from approximately 23,000 to around 10,000 within the first 100 iterations, on the other hand AGWO, at the beginning it has high initial distance of approximately 26,000 but gradually start descending across the iterations, This is a direct result of its adaptive sigmoid-based decay of the parameter a , which intentionally maintains high exploration for longer before shifting toward exploitation — unlike standard GWO which decreases a linearly. Another factor is its hierarchical mechanism. The Alpha, beta and delta structure direct the search towards better solutions, it also shows better balancing to the exploration and exploitation phases as the iterations increase. In addition, none of the algorithms were able to close the gap to the optimum line without the help on the 2-opt post-processing step.

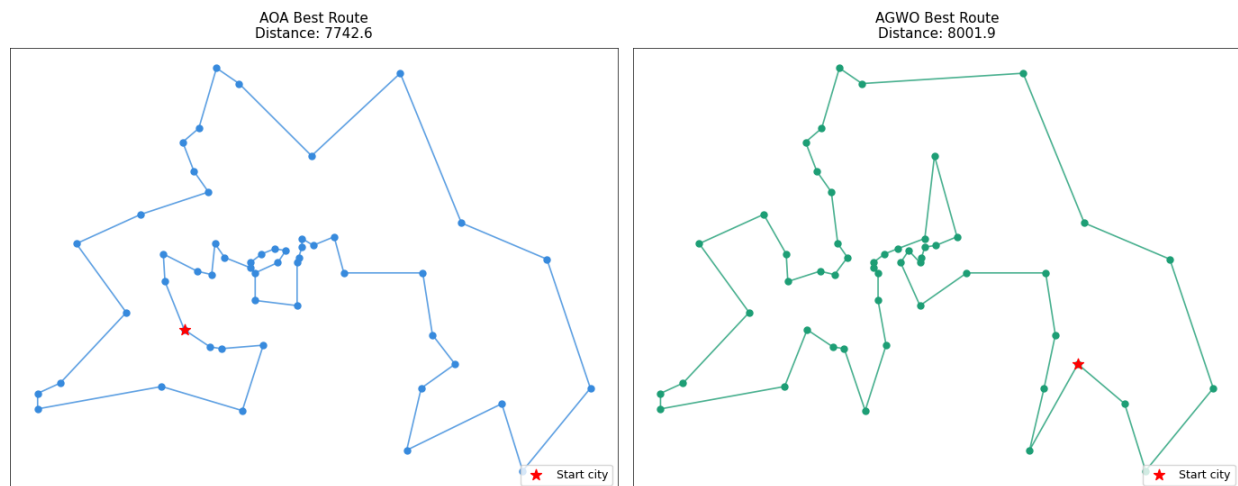


Figure 4: Route Visualization – AOA vs AGWO (berlin 25)

Route Quality (Figure 4): The route visualizations confirm the already found quantitative results. The AOA’s best route (7742.6) has a clean structured path than AGWO’s best route (8001.9) which is a difference of 259.3 units (3.3% longer), which visible crossings around the dense cluster of central cities in in the berlin52 layout. These crossings are a common artefact of AGWO’s crossover-based position update, which can introduce non-planar segments that a pure metaheuristic does not always resolve. The AOA route benefits its perturbation-based operators, guiding the path closer to the optimum solution.

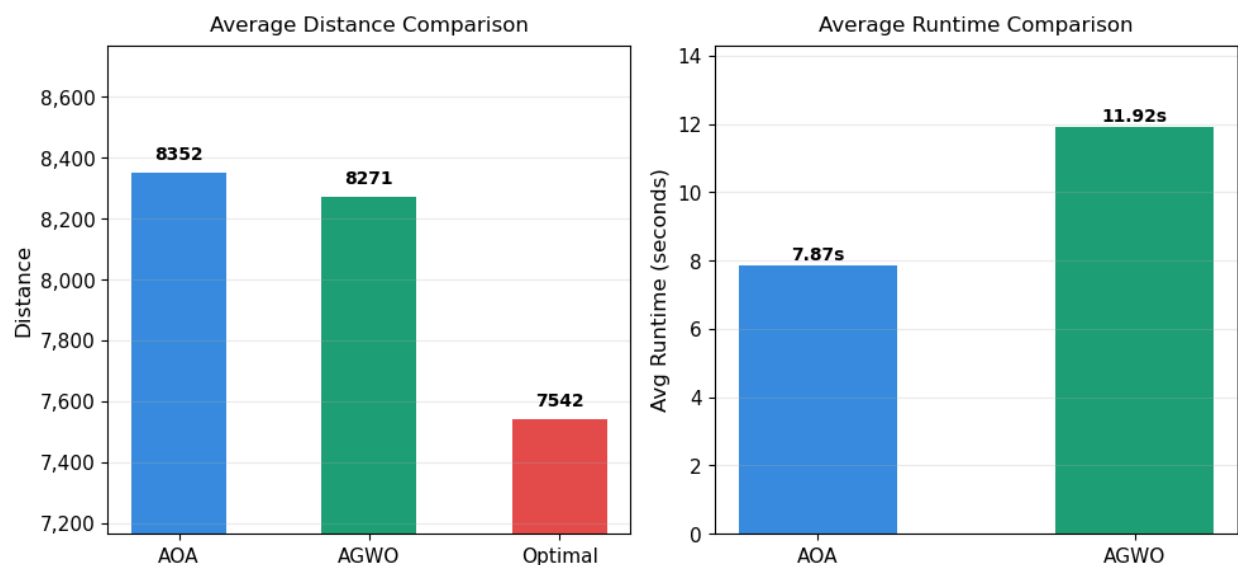


Figure 5: Performance comparison– AOA vs AGWO (berlin 25)

Distance and Runtime Trade-off (Figure 5): this figure presents a direct comparison of average distance and average runtime across all 10 runs; we can clearly see the trade-off between

quality and the computational cost. The left chart confirms that even though AOA found the single best route overall, AGWO produced slightly shorter routes with a mean distance of 8,271 compared to AOA's 8,352, with both remaining above the known optimal of 7,542. As AOA was able to reach a **10.74% (average) optimality gap** in only **7.87 seconds**, while AGWO reached a better **(average) optimality gap of 9.66 %** in **11.92 seconds**. This happens because AOA uses a simpler mathematical framework, which gives it the speed advantage, though this comes at the cost of higher variability. making it preferable for time-sensitive logistics applications where a ‘good’ solution is needed quickly, but where repeatability is critical, AGWO’s lower standard deviation is preferable.

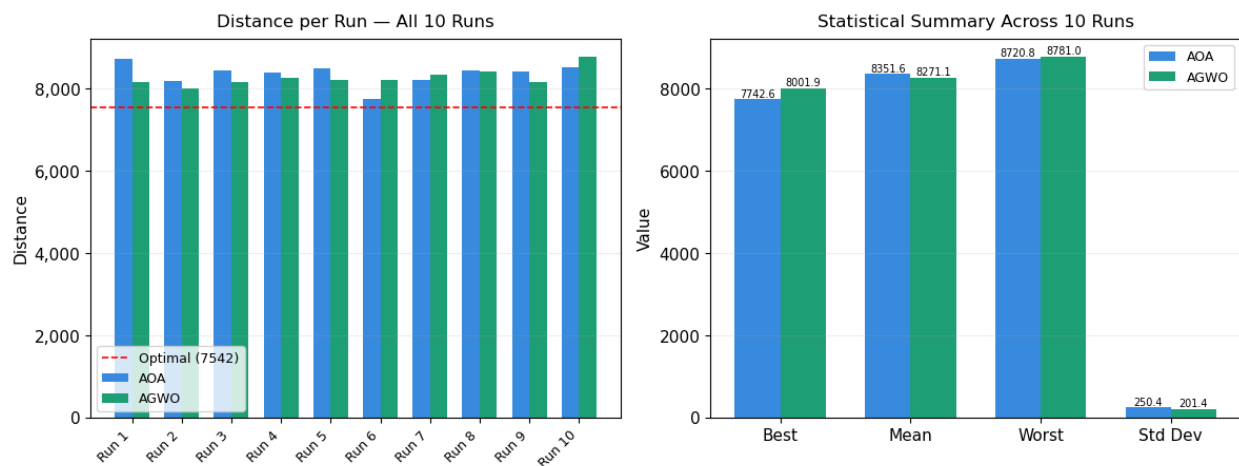


Figure 6: Multiple runs comparison – AOA vs AGWO (berlin 25)

Statistical Robustness (Figure 6): this graph clearly shows both algorithm's performance stability across 10 independent runs, The left chart shows the distance recorded per run, where AGWO's bars cluster more tightly around its mean while AOA displays a wider spread, with some runs performing noticeably better or worse than others. The right graph shows that AGWO standard deviation was 201.4 which is **19.6% less** than AOA’s 250.4 which confirms the consistency of AGWO. AGWO’s Alpha–Beta–Delta hierarchy, acts as a multi-layered filter: even when individual wolves explore suboptimal regions, the ensemble effect of three guiding positions prevents the population from collapsing onto a poor local optimum. In a real-world logistics context, this consistency is operationally significant.

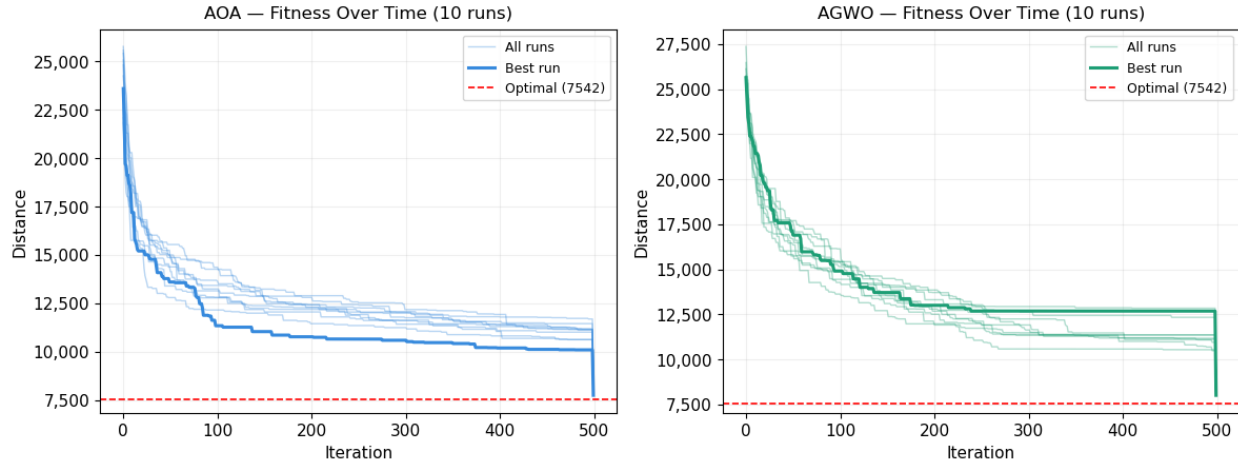


Figure 7: *Fitness overtime – AOA vs AGWO (berlin 25)*

Fitness Over Time (Figure 7): this graph shows how each algorithm behaves during the search, For AOA, there is a widespread between best and worst runs, reflecting AOA's higher standard deviation of 250.4. while AGWO's runs are more grouped, particularly after iteration 200, confirming that the adaptive Alpha–Beta–Delta hierarchy combined with the stagnation restart mechanism provides a stabilizing effect during the later exploitation phase — preventing individual runs from widely diverging from the group. The sharp drop visible at iteration 500 in both graphs represents the 2-opt post-processing step applied at termination, Since the metaheuristic search alone couldn't quite hit the **7,542 optimal marks**.

5.4 Limitations of the Results

Despite producing a comprehensive result and showing a solid comparison between the AOA and AGWO, we still must acknowledge the limitations of our results:

We conducted this experiment using a single benchmark instance which is berlin52 which has only 52 cities and this limits how far we can generalize our findings since it's drawn from a small-scale testing alone and might differ on larger TSPLIB instances like kroA100.

Our chosen algorithms AOA and AGWO are both metaheuristics which means convergence is not guaranteed and might never find the real global optimum, the results show the heuristic approximations and not the proved optimal solutions.

The performance of both algorithms is sensitive to hyperparameter settings, different configuration for the population size (30), the number of iterations (500) or for the adaptive parameters in AGWO (like the convergence factor a)

may produce different results, making it difficult to claim that our chosen configuration is optimal.

Since we used the 2-opt post-processing step at termination to improve both algorithms' final results and maximize the performance it became slightly more difficult to isolate the pure performance of each search mechanism from the final local search refinement.

The Runtime measurements were recorded on one machine and may change when different hardware is used or larger-scale deployment.

5.5 Possible Improvements

In this section we will provide some possible improvements that will minimize the gap between our current results and the known optimal route of **7,542**, several strategic adjustments can be made:

Creating a Hybrid framework: the most effective improvement step is to combine the strengths of our chosen algorithms, AOA uses very fast math operators to "scout" the full map in the first 20% of the iterations which will spread our solutions wide, then we will use the hunting hierarchy of AGWO for the rest 80% to fine-tune the found results, using this approach will fix the inconsistency in AOA while avoiding AGWO finding a "good enough" route too early

Move 2-opt and 3-opt Inside the Loop: instead of only using the **2-opt heuristic** at the very end we can integrate it into the algorithm, like every 50 or 70 iterations this will reduce crossing paths while the algorithm is still searching, because as we can see in the map there are some clusters that could be fixed on the early stage shaving up to 5-10% of the total distance.

Improving the Fine-Tuning and Stress Testing: we can improve our results by precisely fine-tuning the population size and control the Constants like MOP and MOA systematic grid search, then we can test these tweaks on larger map like **kroA100** which has 100 cities, by doing this we will get more reliable results if the map is larger.

Using Opposition-Based Learning (OBL): To prevent AOA from being a hit-or-miss and to reduce the 250.4 standard deviation, we can use OBL at the start. Instead of just randomly picking the routes, for every generated route we also check at its mathematical opposite by keeping the better one. We effectively double our search coverage from the second one without making the code much more complex, Similarly, applying OBL to AGWO's initialization would reduce its reliance on the stagnation restart mechanism to recover from poor starting positions.

6. Conclusion

This study presents a comparative analysis of the Arithmetic Optimization Algorithm (AOA) and Adaptive Grey Wolf Optimizer (AGWO) applied to the Traveling Salesman Problem. AOA achieved superior solution quality (7,742.6 units, 2.66% optimality gap) with faster runtime (7.87 seconds), while AGWO demonstrated greater consistency (6.10% optimality gap, 19.6% lower variance). This fundamental trade-off between speed and reliability has practical implications: AOA suits time-sensitive logistics where rapid solutions are needed, while AGWO is preferable when operational consistency is critical. Neither algorithm guarantees optimality, yet both effectively demonstrate that metaheuristics can solve NP-hard problems to near-optimal quality within reasonable computational time.

However, several limitations constrain generalizability. This evaluation used only the berlin52 benchmark (52 cities); larger instances like kroA100 may reveal different performance patterns. Algorithm performance is sensitive to hyperparameter configuration (population size, iterations, adaptive parameters), and both algorithms benefited from 2-opt post-processing, making it difficult to isolate pure search mechanism performance. The results represent high-quality approximations rather than proven optimal solutions.

Future work should extend evaluation to larger problem instances, explore hybrid frameworks combining both algorithms' strengths, and conduct systematic hyperparameter optimization. Rather than declaring a universal winner, this research demonstrates that algorithm selection should be based on specific problem characteristics and operational constraints. As logistics efficiency remains critical for profitability and sustainability, continued advancement in comparative metaheuristic research is essential for delivering practical optimization solutions at scale.

References:

- Bladimir Toaza, & Domokos Esztergár-Kiss. (2023). A review of metaheuristic algorithms for solving TSP-based scheduling optimization problems Image 1. *Applied Soft Computing*, 148, 110908. <https://doi.org/10.1016/j.asoc.2023.110908>
- De, L., & Di Summa, G. (n.d.). *Methods and Models for Combinatorial Optimization Exact methods for the Traveling Salesman Problem*.
<https://www.math.unipd.it/~luigi/courses/metmodoc1718/m08.01.TSPexact.en.pdf>
- Laith Abualigah, Diabat, A., Seyedali Mirjalili, Elaziz, M. A., & Gandomi, A. H. (2021). The arithmetic optimization algorithm. *Computer Methods in Applied Mechanics and Engineering*, 376, 113609. <https://doi.org/10.1016/j.cma.2020.113609>
- Thirugnanasambandam, K., Raghav, R. S., Saravanan, D., U, P., & M, R. (2019). Experimental analysis of ant system on travelling salesman problem dataset TSPLIB. *EAI Endorsed Transactions on Pervasive Health and Technology*, 163092.
<https://doi.org/10.4108/eai.13-7-2018.163092>
- Vidal, T. (2022). Hybrid genetic search for the CVRP: Open-source implementation and SWAP* neighborhood. *Computers & Operations Research*, 140, 105643–105643.
<https://doi.org/10.1016/j.cor.2021.105643>
- Xu, Z., & Zhang, X. (2024). An Improve Grey Wolf Optimizer Algorithm for Traveling Salesman Problems. *IAENG International Journal of Computer Science*, 51(6), 602–612.
https://www.iaeng.org/IJCS/issues_v51/issue_6/IJCS_51_6_05.pdf